

Lesson 2: Routing Basics

□ Table of Contents

1. [Introduction to Routing](#)
 2. [HTTP Route Methods](#)
 3. [Route Parameters](#)
 4. [Named Routes](#)
 5. [Route Groups](#)
 6. [Route Model Binding](#)
 7. [Practical Exercises](#)
-

Introduction to Routing

Laravel's routing system is the **application's gateway**. Every request to your application passes through the routing system, which decides how to handle it.

Why is Routing Important?

- □ **Organizes application requests:** Links URLs to appropriate logic
- □ **Provides security:** Add middleware for protection
- □ **Easier maintenance:** All routes in one place
- □ **Supports RESTful APIs:** Build professional APIs

Where are Routes Located?

```
routes/  
├─ web.php      # Web routes (HTML pages)  
├─ api.php      # API routes (JSON responses)  
├─ console.php  # Artisan commands  
└─ channels.php # Broadcasting channels
```

HTTP Route Methods

Laravel supports all basic HTTP Methods.

1. GET - Read Data

Used to display data only (doesn't modify database).

```
Route::get('/users', function () {  
    return 'List of users';  
});  
  
Route::get('/user/{id}', function ($id) {  
    return "Show user: $id";  
});
```

When to use?

- Display a page
- Read data
- Search

2. POST - Create New Data

Used to send new data to the server.

```
Route::post('/users', function () {  
    // Create new user  
    return 'User created';  
});
```

When to use?

- Submit a form
- Create new record
- Upload file

3. PUT/PATCH - Update Existing Data

```
// PUT - Full update  
Route::put('/users/{id}', function ($id) {  
    return "Update user $id completely";  
});
```

```
});

// PATCH - Partial update
Route::patch('/users/{id}', function ($id) {
    return "Update some fields of user $id";
});
```

Difference between PUT & PATCH:

- **PUT:** Full update (all fields)
- **PATCH:** Partial update (some fields)

4. DELETE - Delete Data

```
Route::delete('/users/{id}', function ($id) {
    return "Delete user $id";
});
```

5. Route Accepting Multiple Methods

```
Route::match(['get', 'post'], '/form', function () {
    return 'Accepts GET and POST';
});

// All Methods
Route::any('/test', function () {
    return 'Accepts any method';
});
```

Methods Summary Table:

Method	Usage	Example
GET	Read/Display	Display products list
POST	Create new	Add new product
PUT	Full update	Update all product data
PATCH	Partial update	Update product price only
DELETE	Delete	Delete product

Route Parameters

Route parameters allow you to capture values from the URL.

1. Required Parameters

```
// Single parameter
Route::get('/user/{id}', function ($id) {
    return "User ID: $id";
});

// Multiple parameters
Route::get('/post/{postId}/comment/{commentId}', function ($postId, $commentId) {
    return "Post $postId - Comment $commentId";
});
```

Example URLs:

- `/user/5` → `$id = 5`
- `/post/10/comment/3` → `$postId = 10, $commentId = 3`

2. Optional Parameters

```
Route::get('/user/{name?}', function ($name = 'Guest') {
    return "Hello $name";
});
```

Examples:

- `/user/John` → `"Hello John"`
- `/user` → `"Hello Guest"`

3. Parameter Constraints (Regular Expressions)

```
// Numbers only
Route::get('/user/{id}', function ($id) {
    return "User: $id";
})->where('id', '[0-9]+');
```

```
// Letters only
Route::get('/user/{name}', function ($name) {
    return "User: $name";
})->where('name', '[A-Za-z]+');

// Multiple constraints
Route::get('/user/{id}/{name}', function ($id, $name) {
    return "ID: $id, Name: $name";
})->where(['id' => '[0-9]+', 'name' => '[a-z]+']);
```

4. Global Constraints

In `app/Providers/RouteServiceProvider.php`:

```
public function boot(): void
{
    // Every {id} must be a number
    Route::pattern('id', '[0-9]+');

    // Every {slug} must be alphanumeric with dashes
    Route::pattern('slug', '[a-z0-9-]+');
}
```

Named Routes

Named routes make it easier to reference routes in your application.

Why Named Routes?

□ **Easy modification:** Change URL without changing code □ **Code clarity:** Clear names instead of URLs □ **Avoid errors:** No need to write URLs manually

Defining Named Routes

```
Route::get('/user/profile', function () {
    return 'User Profile Page';
})->name('profile');

Route::get('/dashboard', function () {
    return 'Dashboard';
});
```

```
})->name('dashboard');
```

Using Named Routes

In Blade Templates:

```
<!-- Simple link -->
<a href="{ route('profile') }}">Profile</a>

<!-- Link with parameters -->
<a href="{ route('user.show', ['id' => 1]) }}">Show User</a>

<!-- Redirect -->
return redirect()->route('dashboard');
```

In Controllers:

```
// Redirect
return redirect()->route('profile');

// With parameters
return redirect()->route('user.show', ['id' => $userId]);

// With message
return redirect()->route('dashboard')
    ->with('success', 'Success!');
```

Getting Route URL:

```
$url = route('profile'); // http://yourapp.com/user/profile
```

Organized Route Naming

```
// Clear naming
Route::get('/posts', [PostController::class, 'index'])->name('posts.index');
Route::get('/posts/{id}', [PostController::class, 'show'])->name('posts.show');
Route::post('/posts', [PostController::class, 'store'])->name('posts.store');
Route::put('/posts/{id}', [PostController::class, 'update'])->name('posts.update');
Route::delete('/posts/{id}', [PostController::class, 'destroy'])->name('posts.destroy');
```

Recommended naming pattern: `resource.action`

Route Groups

Route groups help organize similar routes.

1. Route Prefix

```
// Without group
Route::get('/admin/users', function () { });
Route::get('/admin/posts', function () { });
Route::get('/admin/settings', function () { });

// With group - Better!
Route::prefix('admin')->group(function () {
    Route::get('/users', function () { });           // /admin/users
    Route::get('/posts', function () { });           // /admin/posts
    Route::get('/settings', function () { });        // /admin/settings
});
```

2. Route Middleware

```
Route::middleware(['auth'])->group(function () {
    Route::get('/dashboard', function () {
        return 'Dashboard';
    });

    Route::get('/profile', function () {
        return 'Profile';
    });
});
```

3. Name Prefix

```
Route::name('admin.')->group(function () {
    Route::get('/dashboard', function () {
        // ...
    })->name('dashboard'); // Full name: admin.dashboard

    Route::get('/users', function () {
        // ...
    })->name('users');     // Full name: admin.users
```

```
});
```

4. Combining All Properties

```
Route::prefix('admin')
    ->middleware(['auth', 'admin'])
    ->name('admin.')
    ->group(function () {

        Route::get('/dashboard', function () {
            return 'Dashboard';
        })->name('dashboard'); // admin.dashboard

        Route::get('/users', function () {
            return 'Users';
        })->name('users');      // admin.users

        Route::get('/posts', function () {
            return 'Posts';
        })->name('posts');      // admin.posts
    });
```

Result:

- URLs: /admin/dashboard, /admin/users, /admin/posts
- Names: admin.dashboard, admin.users, admin.posts
- Middleware: auth, admin on all routes

Route Model Binding

Powerful feature that allows Laravel to automatically fetch models from the database.

1. Implicit Binding (Automatic)

```
use App\Models\User;

// Without Model Binding
Route::get('/user/{id}', function ($id) {
    $user = User::findOrFail($id);
    return view('user.profile', ['user' => $user]);
});
```



```
// With Model Binding - Simpler!
Route::get('/user/{user}', function (User $user) {
    // Laravel fetches the user automatically
    return view('user.profile', ['user' => $user]);
});
```

How does it work?

1. Laravel notices the parameter name is {user}
2. Notices the type is User \$user
3. Searches for User::find(\$id) automatically
4. Returns 404 if not found

2. Custom Key

By default, Laravel searches by `id`. But you can change it:

```
// Search by slug instead of id
Route::get('/post/{post:slug}', function (Post $post) {
    return view('post.show', ['post' => $post]);
});
```

Example URL: /post/laravel-routing-tutorial

3. Customization in Model

In App\Models\Post.php:

```
public function getRouteKeyName()
{
    return 'slug'; // Use slug instead of id
}
```

Now you can:

```
Route::get('/post/{post}', function (Post $post) {
    // Searches by slug automatically
    return view('post.show', ['post' => $post]);
});
```

Redirects

Redirect Types:

```
// 1. Simple redirect
Route::get('/old-page', function () {
    return redirect('/new-page');
});

// 2. Redirect to named route
Route::get('/home', function () {
    return redirect()->route('dashboard');
});

// 3. Permanent redirect (301)
Route::redirect('/old', '/new', 301);

// 4. Redirect back
return back();

// 5. Redirect with data
return redirect()->route('dashboard')
    ->with('success', 'Success!');
```

Fallback Route

Fallback route when no route matches:

```
Route::fallback(function () {
    return view('errors.404');
});
```

View All Routes

```
php artisan route:list
```

Useful options:

```
# Specific routes only
php artisan route:list --path=api

# Search
php artisan route:list --name=user

# With middleware
php artisan route:list --middleware=auth
```

Best Practices

☐ Do:

1. Use clear names

```
Route::get('/posts', [PostController::class,
'index'])->name('posts.index');
```

2. Organize routes in groups

```
Route::prefix('admin')->group(function () {
    // Admin routes
});
```

3. Use Resource Routes for CRUD

```
Route::resource('posts', PostController::class);
```

4. Use Model Binding

```
Route::get('/user/{user}', function (User $user) {  
    return view('user', compact('user'));  
});
```

❌ Don't:

1. Don't put logic in routes

```
// ❌ Bad  
Route::get('/users', function () {  
    $users = User::all();  
    // 50 lines of code...  
});  
  
// ✅ Good  
Route::get('/users', [UserController::class, 'index']);
```

2. Don't repeat the same code

```
// ❌ Bad  
Route::get('/admin/users', ...)->middleware('auth');  
Route::get('/admin/posts', ...)->middleware('auth');  
  
// ✅ Good  
Route::middleware('auth')->prefix('admin')->group(...);
```

Practical Exercises

Exercise 1: Basic Routes ✅

Create the following routes in `routes/web.php`:

```
// 1. Home page  
Route::get('/', function () {
```

```

        return view('welcome');
    });

    // 2. About page
    Route::get('/about', function () {
        return view('about');
    });

    // 3. Contact page
    Route::get('/contact', function () {
        return view('contact');
    });

```

Exercise 2: Route Parameters

```

// 1. Show product by ID
Route::get('/product/{id}', function ($id) {
    return "Show product: $id";
})->where('id', '[0-9]+');

// 2. Show product by slug
Route::get('/product/{slug}', function ($slug) {
    return "Show product: $slug";
})->where('slug', '[a-z0-9-]+');

// 3. Multiple parameters
Route::get('/category/{category}/product/{product}', function ($category, $product)
{
    return "Category: $category - Product: $product";
});

```

Exercise 3: Named Routes

```

Route::get('/dashboard', function () {
    return view('dashboard');
})->name('dashboard');

Route::get('/profile', function () {
    return view('profile');
})->name('profile');

// In Blade:
// <a href="{{ route('dashboard') }}">Dashboard</a>

```

Exercise 4: Route Groups

```
// Admin routes group
Route::prefix('admin')->name('admin.')->middleware('auth')->group(function () {
    Route::get('/dashboard', function () {
        return 'Admin Dashboard';
    })->name('dashboard');

    Route::get('/users', function () {
        return 'Manage Users';
    })->name('users');

    Route::get('/posts', function () {
        return 'Manage Posts';
    })->name('posts');
});
```

Exercise 5: Different HTTP Methods

```
// Contact form
Route::get('/contact', function () {
    return view('contact');
})->name('contact.show');

Route::post('/contact', function () {
    // Process data
    return redirect()->route('contact.show')
        ->with('success', 'Message sent!');
})->name('contact.submit');
```

Challenge: Complete CRUD System

```
// Posts CRUD
Route::get('/posts', function () {
    return 'Posts list';
})->name('posts.index');

Route::get('/posts/create', function () {
    return 'Create new post';
})->name('posts.create');

Route::post('/posts', function () {
    return 'Store post';
})->name('posts.store');
```

```
Route::get('/posts/{id}', function ($id) {
    return "Show post $id";
})->name('posts.show');

Route::get('/posts/{id}/edit', function ($id) {
    return "Edit post $id";
})->name('posts.edit');

Route::put('/posts/{id}', function ($id) {
    return "Update post $id";
})->name('posts.update');

Route::delete('/posts/{id}', function ($id) {
    return "Delete post $id";
})->name('posts.destroy');
```

□ Summary

In this lesson, you learned:

□ HTTP Methods (GET, POST, PUT, DELETE) □ Route parameters (required and optional) □ Parameter constraints (Regular Expressions) □ Named routes and their benefits □ Route groups and organization □ Route Model Binding □ Routing best practices

□ Additional Resources

- [Laravel Routing Documentation](#)
- [RESTful Resource Controllers](#)
- [Route Model Binding](#)

□ Test Yourself

Before moving to Lesson 3, make sure you can answer:

1. What's the difference between GET and POST?

2. How do you create an optional route parameter?
 3. Why do we use named routes?
 4. How do you create a route group with a prefix?
 5. What is Route Model Binding?
-

Next Lesson

Ready for more? Move on to [Lesson 3: Controllers and MVC Pattern](#)

In Lesson 3, you'll learn:

- Creating Controllers
 - Organizing application logic
 - Resource Controllers
 - Dependency Injection
 - And more!
-

Happy Learning! ☐